NAME	ID
احمد سامر السيد (leader)	21033990
اسلام محمد رجب	21088622
مصطفي عبدالرحمن رمضان	21122808
عبدالرحمن ايهاب صلاح	21033945
يوسف محمد عبدالتواب	21015239
محمد محمود يوسف	21013277

System Analysis & Design

تحليل وتصميم النظام

GlobeClass — AI-Powered Satellite Image Land Cover Classification

1. System Architecture

GlobeClass follows a client-server architecture with a clear separation between the frontend (presentation layer) and backend (application + ML inference layer).

Architecture Overview:

- Client (Browser): HTML/CSS/JS frontend with Three.js 3D globe, Leaflet.js satellite maps, Chart.js visualizations, and i18n bilingual support
- Server (Flask on Railway): REST API handling image classification, satellite capture, model management, and static file serving
- External Services: ESRI ArcGIS (satellite imagery), Nominatim/OSM (geocoding), GitHub (model storage)

Key Design Decisions:

- Single-model memory policy: Only one model loaded at a time to prevent OOM on cloud deployment
- Lazy model loading: Models are loaded on first request, not at startup
- INT8 dynamic quantization on CPU for reduced memory and faster inference
- Frontend served by Flask (not separate hosting) for simplified deployment

2. API Routes

Method	Route	Purpose
GET	/	Serve landing page
GET	/classify	Serve Observatory classification page
GET	/health	Health check endpoint
GET	/api/model-info	Return metadata for all available models
GET	/api/sample-images	Return base64-encoded test sample images
GET	/api/satellite-capture	Fetch satellite image from ESRI for given coordinates
POST	/api/classify	Classify uploaded image or base64 image using selected model

3. Backend Class Design

ModelManager (Singleton Pattern)

- configs{}: Dictionary mapping model IDs to metadata (name, path, input_size, accuracy, type)
- models{}: Dictionary of loaded model instances (max 1 at a time)
- get_model(id): Lazy loads the requested model, applies quantization, evicts previous model
- get_transform(size): Returns torchvision.transforms.Compose pipeline for preprocessing
- _build_resnet(): Constructs ResNet50 with custom classification head

Model Architectures:

- ResNet50: torchvision.models.resnet50 with custom FC head (Dropout→Linear→ReLU→Dropout→Linear)
- SwinClassifier: timm swin_base_patch4_window7_224 backbone + LayerNorm→Dropout→Linear→GELU→Dropout→Linear head

- PrithviClassifier: terratorch prithvi_swin_B backbone with Conv2d(3→6) channel adapter + AdaptiveAvgPool2d + classifier head

4. Sequence Diagram — Image Classification Flow

1. User uploads image or captures satellite view
2. Frontend sends POST /api/classify with image data + model ID
3. Flask API calls ModelManager.get_model(model_id)
4. ModelManager lazy-loads model weights, applies INT8 quantization, evicts previous model
5. API preprocesses image: resize → tensor → normalize
6. Model performs forward pass: model(tensor)
7. API applies softmax to get 10-class probabilities
8. API returns JSON: {success, prediction, confidence, results[10]}
9. Frontend displays results: primary card, breakdown list, Chart.js bar chart
10. Entry added to session-based classification history

5. Data Flow Diagram

Input Sources:

- File Upload: User drops/selects JPG/PNG image → image bytes
- Satellite Capture: User navigates map → ESRI API fetches 512×512 PNG → base64 → image bytes
- Sample Gallery: Pre-encoded test images from EuroSAT dataset → base64 → image bytes

Processing Pipeline:

- Image bytes → PIL Image (RGB) → Resize to model input size → ToTensor → Normalize([0.5,0.5,0.5])
- Tensor → Neural Network inference → Raw logits → Softmax → 10 probabilities
- Probabilities → Sort descending → Enrich with class metadata (label, icon, color, description)

Output Destinations:

- Primary Result Card: Top class with icon, label, description, confidence bar
- Full Breakdown: Ranked list of all 10 classes with colored progress bars
- Chart.js Bar Chart: Interactive confidence distribution visualization
- Classification History: Session log with timestamps

6. Database Design

Note: GlobeClass does not use a traditional relational database. All data is either file-based (model weights, images) or session-based (classification history in browser memory). This section documents the data structures used.

6.1 Model Configuration (In-Memory Dictionary)

Field	Type	Example
id	string	resnet50
name	string	ResNet50 (Transfer Learning)
path	string	Models/best_model.pth
input_size	tuple(int,int)	(64, 64)
accuracy	string	98.25%
type	string	resnet
desc	string	Balanced performance and speed.

6.2 Classification Result (JSON Response)

Field	Type	Example
success	boolean	true

prediction	string	Forest
confidence	float	97.82
results	array[10]	[{class_id, label, icon, color, description, confidence}]

6.3 File-Based Storage

Data	Location	Format
ResNet50 weights	Models/best_model.pth	PyTorch state_dict (~92 MB)
Swin-B weights	Models/swin_best_model.pth	PyTorch state_dict (~333 MB)
Prithvi-EO weights	Models/prithvi_best_model.pth	PyTorch state_dict (~334 MB)
Training data	Data EuroSAT_split_data_TVT/	27K images in class folders
Raw dataset	Data/ EuroSAT RGB dataset/	Original unsplit dataset

7. Technology Stack

Layer	Technology	Version	Justification
ML Framework	PyTorch	2.3.1+ (CPU)	Industry standard; native support for all 3 model architectures
Model Library	timm	0.9.16	Provides 700+ pre-trained architectures including Swin Transformer
Foundation Model	terratorch	Latest	IBM's library for Prithvi-EO model registration

Backend	Flask	3.0.3	Lightweight, easy REST API development
CORS	Flask-CORS	4.0.1	Cross-origin request handling
Server	Gunicorn	22.0.0	Production WSGI server with timeout configuration
Image Processing	Pillow	10.4.0	Image loading and format conversion
HTTP Client	requests	2.32.3	ESRI satellite image fetching
3D Rendering	Three.js	0.160.0	WebGL 3D Earth globe with textures
Mapping	Leaflet.js	1.9.4	Interactive satellite map with ESRI tiles
Charts	Chart.js	4.4.4	Confidence distribution visualization
CSS Framework	Tailwind CSS	CDN (latest)	Rapid responsive UI development
Fonts	Inter, Manrope	Google Fonts	Modern typography
Icons	Material Symbols	Google	Consistent icon system
Deployment	Railway	Cloud PaaS	Automatic builds from Git, Procfile-based
VCS	Git / GitHub	-	Source control and collaboration

8. Interface Design

8.1 Landing Page

- Top Navigation: Brand logo, Explore, Observatory, Launch Lab button, language toggle
- Hero Section: Title ("Behold the Infinite"), subtitle, animated 3D globe, CTA buttons
- Neural Architectures: Interactive card carousel showing ResNet50, Swin Transformer, Prithvi-EO details
- How It Works: Three-step visual pipeline (Upload → Neural Classification → Results)
- Land Cover Classes: Grid of 10 class cards with icons and descriptions
- Model Stats: Tabbed section comparing all three architectures with training phases
- Footer: Navigation links, copyright

8.2 Observatory (Classification) Page

- Top Nav: Brand, page links, model status badge, mobile menu
- Left Sidebar (desktop): Classification HUD, view mode toggle (Globe/Satellite), search, coordinates, model selector
- Main Viewport (8-column): 3D globe with coordinate overlay OR Leaflet satellite map with ESRI tiles
- Upload Zone: Drag-and-drop area with preview, classify, and clear buttons
- Results Panel (4-column): Captured image preview, primary result card, full breakdown, Chart.js chart, classification history log, sample gallery
- Mobile: Collapsible toolbar, responsive stacked layout, touch-optimized controls

9. Model Architecture Details

9.1 ResNet50 (Transfer Learning)

Property	Value
Base Architecture	ResNet50 (ImageNet pre-trained)
Input Size	64 × 64 × 3 (RGB)
Total Parameters	~24 Million
Training Strategy	Phase 1: Backbone frozen (feature extraction) → Phase 2: Full fine-tuning

Custom Head	Dropout(0.4) → Linear(2048, 256) → ReLU → Dropout(0.3) → Linear(256, 10)
Test Accuracy	98.25%

9.2 Swin Transformer (Base)

Property	Value
Base Architecture	swin_base_patch4_window7_224
Input Size	224 × 224 × 3 (RGB)
Total Parameters	~88 Million
Training Strategy	Shifted-window self-attention, hierarchical patch merging
Custom Head	LayerNorm(1024) → Dropout(0.3) → Linear(1024, 512) → GELU → Dropout(0.2) → Linear(512, 10)
Test Accuracy	98.42%

9.3 Prithvi-EO 2.0 Foundation Model

Property	Value
Base Architecture	Prithvi Swin-B (IBM/NASA)
Input Size	224 × 224 × 3 (RGB → 6-band via Conv2d adapter)
Total Parameters	~87 Million
Training Strategy	MAE self-supervised pre-training → full fine-tuning on EuroSAT
Channel Adapter	Conv2d(3, 6, kernel_size=1) — projects RGB to 6-band HLS-like input

Custom Head	AdaptiveAvgPool2d(1) → LayerNorm → Dropout(0.3) → Linear(dim, 512) → GELU → Dropout(0.2) → Linear(512, 10)
Test Accuracy	91.23%

10. Deployment Architecture

Component	Configuration
Procfile	web: gunicorn -w 1 -b 0.0.0.0:$PORT app:app --timeout 120
Python	3.10 (Railway default)
PyTorch	CPU-only wheels (linux_x86_64)
Model Loading	Lazy loading with single-model memory policy
Optimization	INT8 dynamic quantization on CPU

External Dependencies:

- ESRI ArcGIS World Imagery: Satellite tile service for live capture
- Nominatim (OpenStreetMap): Geocoding API for place name search
- GitHub (LFS): Model weight file storage and versioning

11. Project Directory Structure

```
GlobeClass/
├── Frontend/                  # Client-side web application
│   ├── index.html             # Landing page (567 lines)
│   ├── classify/index.html    # Observatory page (539 lines)
│   ├── manifest.json          # PWA manifest
│   ├── service-worker.js      # PWA caching service worker
│   └── static/                # CSS, JS, images
├── Notebooks/                 # Training scripts
│   ├── notebook milestone_1_2.ipynb  # Training notebook
│   └── prithvi_model.py       # Prithvi training script (375 lines)
├── Models/                    # Trained model weights (.pth)
├── Data/                      # Original EuroSAT RGB dataset
├── Data EuroSAT_split_data_TVT/ # Train/Valid/Test split
├── deployment/railway_backend/  # Flask API + Procfile
├── GlobeClass_models/         # Separate model repository
├── requirements.txt           # Root project dependencies
└── docs/                      # Project documentation
```